POLYMATH-ATLAS / OSA-4.0 АРХИТЕКТУРНЫЙ КОДЕКС

От хаоса лексем к формальному семантическому ядру. Полная хронология синтеза.

ПРОЛОГ: ИСХОДНОЕ СОСТОЯНИЕ И КРИЗИС

Проект начался с амбициозной попытки формализовать универсальный цикл познания и инженерии через каталог из 418 русских глаголов (типизация, классификация, формализация и т.д.). Однако первичный аудит выявил фундаментальный дефект: смешение в одном списке операций, результатов, свойств, артефактов и дисциплин. Линейная цепочка

```
Целое → Выделить → Различить → ... → Новое Целое
```

оказалась лишь одним из возможных workflow-паттернов, а не универсальным законом.

РЕМАРКА АУДИТОРА: *Отвергнуто: попытка построить операционную алгебру напрямую из естественного языка. Словарь без строгой селекции — свалка. Лингвистический детерминизм порождает этноцентрическую онтологию.*

ФАЗА 1: АРХИТЕКТУРНЫЙ СДВИГ (S-1.0)

Вместо одной иерархии L0-L4 введены **5 ортогональных осей**:

- **Ось A (Категория):** Object, Data, Model, Decision, Action, Meta.
- **Ось B (Роль):** Input, Output, Resource, Target, Constraint.
- **Ось C (Уровень абстракции):** L0 (Reality) → L1 (Data) → L2 (Model) → L3 (Decision) → L4 (Action).
- **Ось D (Тип):** Формальная решётка с правилами IsSubtype(A, B).
- **Ось E (Фаза):** Created, Available, Consumed, Destroyed.

ФАЗА 2: КОНТРАКТНАЯ СИСТЕМА (OSA-3.X)

Введен строгий операторный контракт, отказ от строковых предусловий и переход к State/Dataflow Engine. Система эффектов разделена на E0 (Pure) - E4 (Irreversible).

РЕМАРКА АУДИТОРА: *Отвергнуто:*

```
destroyed ∈ state ⇒ violation
```

. Оператор DELETE должен уничтожать данные. Ошибка возникает только если уничтоженный ресурс нужен достижимому продолжению

графа (Liveness Analysis).

ФАЗА 3: СЕМАНТИЧЕСКОЕ ЯДРО V0.4 (ALGEBRAIC CLOSURE)

Окончательный переход от эвристик к чистой математике. Внедрены 3-значная логика, Weakest Precondition (WP) трансформер и алгебра идентичности.

1. ОБРАТНЫЙ ТРАНСФОРМЕР ПРЕДИКАТОВ (WP)

Анализ живучести переведен с уровня FactID на уровень предикатов. Используется логика Хоара:

```
LiveBefore(Op, LiveAfter) = Requires(Op) ∪ WP(Op, LiveAfter)
WP(Op, Q) = ⋀_{P ∈ Q} ( (Produces(Op) ⟹ P) ∨ P ) \
Invalidates(Op)
```

Если оператор производит

```
VerifiedCalibration(X)
```

, а требуется

```
Calibrated(X)
```

, WP выводит импликацию, и требование удовлетворяется математически, а не субъективно.

2. EPISTEMIC JOIN-SEMILATTICE (АЛГЕБРА СЛИЯНИЯ ВЕТВЕЙ)

Строгое замыкание таблицы истинности для слияния ветвей графа.

```
CONFLICT
```

— топовый элемент.

⊔	TRUE	FALSE	UNKNOWN	ABSENT	CONFLICT
TRUE	TRUE	CONFLICT	UNKNOWN	UNKNOWN	CONFLICT
FALSE	CONFLICT	FALSE	UNKNOWN	UNKNOWN	CONFLICT
UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	UNKNOWN	CONFLICT
ABSENT	UNKNOWN	UNKNOWN	UNKNOWN	ABSENT	CONFLICT
CONFLICT	CONFLICT	CONFLICT	CONFLICT	CONFLICT	CONFLICT

РЕМАРКА АУДИТОРА: *Доказаны идемпотентность, коммутативность и ассоциативность. ABSENT (объекта нет в ветви) ≠ FALSE (доказано, что предикат ложен).*

3. АЛГЕБРА ИДЕНТИЧНОСТИ И РЕКОНСТРУКЦИИ

Запрещен

```
RESTORE(X)
```

как неоднозначный примитив. Введено строгое разделение:

- **Resurrection (Запрещен):** Destroyed(X) → Available(X).
- **Reconstruction (Разрешен):** Порождает X' с новым ID и Provenance.

4. LEXICOGRAPHIC REPAIR OPTIMIZER

Оптимизация ремонта использует лексикографический порядок, где безопасность имеет абсолютный приоритет над стоимостью.

```
Вектор V = [Safety Violations, Residual Proof Obligations, Effect Policy, Uncertainty, Cost]
V1 = [1, 0, 0, 0, 1] (Unsafe, Cheap)
V2 = [0, 0, 0, 0, 3] (Safe, Expensive)
Выбор: V2 (т.к. v1(V2) < v1(V1))
```

ФАЗА 4: ЗАКОНЫ ЯДРА (KERNEL LAWS V0.4)

17 аксиом, готовых к трансляции в Lean 4 / Coq:

1. **Type Reflexivity:** $\forall A \in T, A \subseteq A$
2. **Type Transitivity:** $A \subseteq B \land B \subseteq C \implies A \subseteq C$
3. **State Conservation:** Если Op не меняет X, то State_after(X) = State_before(X)
4. **No Resurrection:** $\nexists$ Op: Produces(X) where State_before(X) = Destroyed без генерации нового ID
5. **Consume Safety:** δ_consume(X) допустима $\iff$ State_before(X) = Available
6. **Predicate Liveness Soundness:** Op с KILL = {P} допустим $\iff P \notin$ LIVE_OUT(Op)
7. **Semilattice Soundness:** Eval_merge(P) = $\sqcup$ Eval_branches(P)
8. **Effect Monoid Associativity:** E(G) инвариантен к порядку группировки
9. **Epistemic Non-Overclaim:** U_out(Op) ≥ U_in(Op) без явного GEN(Evidence)

10. **Composition Soundness:** A ; B допустима ⟺ S ⟶ S' и Eval(Precond(B)) ∉ {FALSE, CONFLICT, ABSENT}
11. **Effect Authorization Soundness:** Effect(Op) ⊨ RequiresAuth ⟹ Auth ∈ LIVE_IN(Op)
12. **WP Transformer Soundness:** {WP(Op, Q)} Op {Q}
13. **Join Associativity:** (x ⊔ y) ⊔ z = x ⊔ (y ⊔ z)
14. **Join Commutativity:** x ⊔ y = y ⊔ x
15. **Join Idempotence:** x ⊔ x = x
16. **Recreation Identity:** RECREATE(X) ⟹ ID(Output) ≠ ID(Input)
17. **Predicate Invalidation Soundness:** Invalidates(Op, P) ⟹ P ∈ KILL(Op)

ФАЗА 5: SEMANTIC TORTURE SET (КРАТКИЕ ИТОГИ)

Ядро прошло проверку минимальными контрпримерами:

- **T03 (Kill Live Fact):** DELETE(X) → ANALYZE(X) = REJECT (Liveness Violation).
- **T05 (Epistemic Unknown):** MEASURE → MODEL (Sufficiency = UNKNOWN) = UNDECIDABLE.
- **T09 (Branch Conflict):** TRUE ⊔ FALSE = CONFLICT = REJECT.
- **T11 (Invalidate Property):** INVALIDATE(Calibrated(X)) → USE(X) = REJECT.

ЭПИЛОГ: ОТКРЫТЫЕ ПРОБЛЕМЫ

Полиматический аудит признает границы текущей формализации. Следующие концепты не могут быть полностью формализованы в рамках ядра и требуют

внешнего оракула или расширения системы:

- **Интерпретация:** Контекст бесконечен, смысл не типизируется. Оставлена как сквозной комбинатор.
- **Модель Агента:** Компетенция и юрисдикция — социальные конструкты, не поддающиеся строгой алгебре.
- **Валидация Цели:** Формальная система не может оценивать этичность или истинность самой цели.

РЕМАРКА АУДИТОРА: *Финальный статус: Polymath-Atlas — это не универсальный алгоритм мышления и не готовое ядро AGI. Это строгая, типизированная операционная семантика, способная строить, проверять и аудировать графы преобразований, честно фиксируя собственные эпистемические границы.*

Compiled by Polymath-Architect | OSA-4.0 v0.4